

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN LAB 2

Môn học: Hệ điều hành

Lớp: 22_4

Giảng viên lý thuyết: Trần Trung Dũng

Giảng viên thực hành: Nguyễn Thanh Quân

Thành viên nhóm:

22120091

22120094

22120213

Phạm Khánh Hân

Lê Bảo Hồng Hạnh

Đoàn Thị Minh Anh

Hồ Chí Minh, ngày 1 tháng 12 năm 2024

MỤC LỤC

A. THÔNG TIN CHUNG:	3
B. NỘI DUNG ĐỒ ÁN	4
1. Using GDB: (trả lời trong answers-syscall.txt.)	4
2. System call tracing:.....	6
3. System call sysinfo:	8
C. TÀI LIỆU THAM KHẢO:	9

A. THÔNG TIN CHUNG:

MSSV	Họ và tên	Phân công công việc
22120091	Phạm Khánh Hân	Tracing
22120094	Lê Bảo Hồng Hạnh	Sysinfo
22120213	Đoàn Thị Minh Anh	Gdb

Câu hỏi 4: Write down the assembly instruction the kernel is panicing at. Which register corresponds to the variable num?

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
scause=0xd sepc=0x80001c82 stval=0x0
panic: kerneltrap
```

```
// num = p->trapframe->a7;
num = * (int *) 0;
80001c82: 00002683          lw  a3,0(zero) # 0 <_entry-0x80000000>
```

- ➔ Lệnh assembly mà kernel đang gặp sự cố là lw a3, 0(zero).
- ➔ Thanh ghi a3 tương ứng với biến num.

Câu hỏi 5: Why does the kernel crash? Hint: look at figure 3-3 in the text; is address 0 mapped in the kernel address space? Is that confirmed by the value in scause above? (See description of scause in [RISC-V privileged instructions](#))

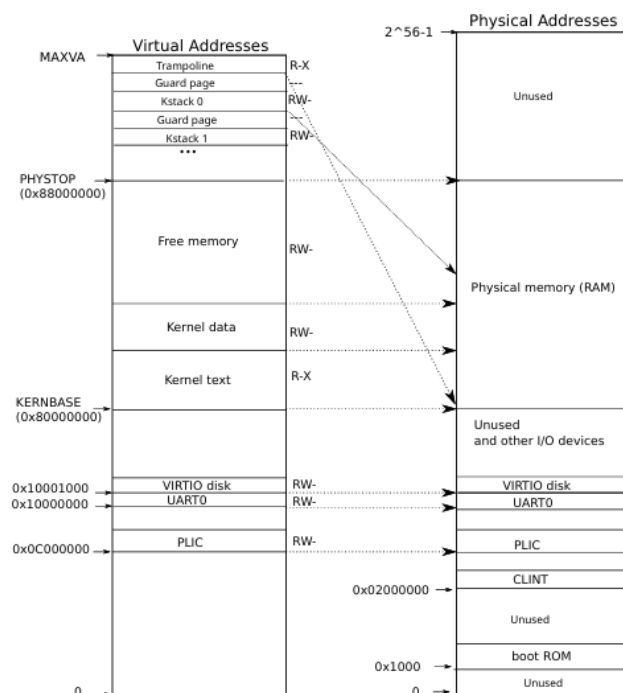


Figure 3.3: On the left, xv6's kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

- ➔ Kernel gặp sự cố do nó cố gắng truy cập vào vùng nhớ không hợp lệ.

- ➔ Theo hình 3.3 này, địa chỉ 0 không được ánh xạ vào không gian kernel. Vì vậy, việc truy cập vào địa chỉ này gây ra lỗi khi kernel cố gắng truy cập một địa chỉ bộ nhớ không hợp lệ.
- ➔ Trước đó ta có `scause = 0xd (13)` có nghĩa là Load page fault dựa theo bảng bên dưới. Load page fault là một lỗi xảy ra khi một tiến trình cố gắng truy cập vào trang bộ nhớ không được tải vào bộ nhớ chính, điều này cũng chứng minh kết luận bên trên.

Interrupt	Exception Code	Description
1	0	<i>Reserved</i>
1	1	Supervisor software interrupt
1	2–4	<i>Reserved</i>
1	5	Supervisor timer interrupt
1	6–8	<i>Reserved</i>
1	9	Supervisor external interrupt
1	10–15	<i>Reserved</i>
1	≥16	<i>Designated for platform use</i>
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10–11	<i>Reserved</i>
0	12	Instruction page fault
0	13	Load page fault
0	14	<i>Reserved</i>
0	15	Store/AMO page fault
0	16–23	<i>Reserved</i>
0	24–31	<i>Designated for custom use</i>
0	32–47	<i>Reserved</i>
0	48–63	<i>Designated for custom use</i>
0	≥64	<i>Reserved</i>

Câu hỏi 6: What is the name of the binary that was running when the kernel panicked?
What is its process id (pid)?

```
(gdb) p p->name
$1 = "initcode\000\000\000\000\000\000\000"
```

- ➔ Tên của tệp nhị phân đang chạy là initcode. (các đoạn \000 phía sau là phần đệm với ý nghĩa để chuỗi có độ dài theo yêu cầu).
- ➔ Process id (pid) của nó là 1.

2. System call tracing:

a. Giới thiệu:

System call trace dùng để hỗ trợ theo dõi các system call cụ thể của một tiến trình và các tiến trình con của nó. Việc theo dõi này có thể giúp ích trong việc gỡ lỗi hoặc phân tích hoạt động của kernel.

b. Mục tiêu:

Sửa đổi kernel xv6 để in ra mỗi dòng thông tin khi có system call bị theo dõi được gọi, lúc này sẽ in ra thông tin gồm: tên system call được gọi và id của tiến trình gọi system call này. Đảm bảo rằng tiến trình con thừa hưởng mask từ tiến trình cha khi fork và giữ nguyên hoạt động của các tiến trình khác không liên quan.

c. Quy trình thực hiện:

- Thêm prototype cho hàm trace trong file user/user.h: để user program có thể gọi hàm.
- Thêm stub trong user/usys.pl để tự động tạo file user/usys.S: để chuyển quyền điều khiển từ user mode vào kernel mode.
- Thêm syscall SYS_trace vào kernel/syscall.h bằng cách gán một số syscall cho trace: vì để kernel biết cần thực hiện chức năng gì. Trong hàm syscall() (được gọi khi có system call), số system call được lấy từ p->trapframe->a7.
- Khai báo một biến để lưu mask bằng cách thêm biến vào struct proc trong file kernel/proc.h: mask sẽ chỉ định những system call nào được theo dõi.
- Viết hàm sys_trace vào file kernel/sysproc.c: sẽ cho phép user chỉ định mask, argint(0,&maskTrace) sẽ lấy tham số mask từ user mode.
- Thêm sys_trace vào bảng syscalls[] trong kernel/syscall.c. (Bảng syscalls[] dùng để ánh xạ số hiệu syscall với hàm xử lý tương ứng trong kernel)
- Sửa hàm fork trong kernel/proc.c để khi một tiến trình gọi fork, tiến trình con phải thừa kế mask từ tiến trình cha.
- Sửa hàm syscall trong kernel/syscall.c, thêm một mảng chứa tên các syscall để in thông tin cho trace. Mảng này chuyển đổi số hiệu syscall thành tên.

d. Lý thuyết phần challenge:

Khi một process gọi system call, tham số được lưu trong các thanh ghi (register) của CPU. Thanh ghi a0 chứa kết quả trả về, a1 đến a5 chứa các tham số của lệnh, và thanh ghi a7 lưu số system call.

Để lấy tham số từ các thanh ghi này trong kernel (khi có một system call), các hàm hỗ trợ sau sẽ được sử dụng:

- **argint(int n, int *ip):** Lấy tham số thứ n được lưu trữ trong các thanh ghi a0, a1, ..., theo thứ tự và lưu vào biến con trỏ ip.
- **argaddr(int n, uint64 *ip):** Lấy tham số thứ n có dạng địa chỉ và lưu vào 1 biến con trỏ.
- **argstr(int n, char *buf, int max):** Lấy tham số thứ n có dạng chuỗi.

Trong kernel của xv6, `fetchstr` là một hàm dùng để lấy chuỗi từ user space về kernel space một cách an toàn: **`fetchstr(uint64 addr, char *buf, int max)`**

- **`uint64 addr`**: địa chỉ này chỉ là một con trỏ chuỗi trong user space
- **`char *buf`**: là bộ đệm (buffer) trong kernel space mà chuỗi sẽ được sao chép vào.
- **`int max`**: là kích thước tối đa mà hàm có thể sao chép từ user space vào kernel space.

3. System call `sysinfo`:

a. Giới thiệu:

System call `sysinfo` được thiết kế để cung cấp thông tin hệ thống liên quan đến bộ nhớ và số lượng tiến trình đang chạy. Việc sử dụng `sysinfo` cho phép người dùng dễ dàng truy xuất các thông tin này từ user mode mà không cần truy cập trực tiếp vào kernel. Đây là một công cụ hữu ích cho việc giám sát và phân tích hoạt động của hệ thống.

b. Mục tiêu:

- Sửa đổi kernel của xv6 để triển khai system call `sysinfo`, cung cấp thông tin về số lượng bộ nhớ trống và số lượng tiến trình đang chạy.
- Đảm bảo rằng system call này có thể được gọi từ user mode và trả về các thông tin một cách chính xác.
- Cung cấp thông tin hệ thống cụ thể để hỗ trợ việc theo dõi và phân tích hoạt động của kernel và tiến trình.

c. Quy trình thực hiện:

- Thêm `$U/_sysinfotest\` vào mục `UPROGS` trong `Makefile`: để đảm bảo rằng chương trình `sysinfotest` được biên dịch và liên kết cùng với các chương trình khác trong hệ thống.
- Khai báo struct `sysinfo` và prototype cho hàm `sysinfo` trong file `user/user.h`: để user program có thể gọi hàm.
- Thêm stub trong `user/usys.pl` để tự động tạo file `user/usys.S`: để chuyển quyền điều khiển từ user mode vào kernel mode.
- Thêm syscall `SYS_sysinfo` vào `kernel/syscall.h` bằng cách gán số hiệu system call cho `sysinfo`: để kernel biết cần thực hiện chức năng gì.
- Thêm hàm handler `sys_sysinfo` bằng `syscalls[]` trong `kernel/syscall.c`: Bảng này ánh xạ các số hiệu system call (như `SYS_sysinfo`) tới các hàm tương ứng (như `sys_sysinfo`), để kernel biết phải gọi hàm nào khi nhận được yêu cầu tương ứng.
- Viết hàm `sys_sysinfo` vào file `kernel/sysproc.c`: hàm này sẽ thực hiện các thao tác cần thiết để thu thập thông tin về bộ nhớ và số lượng tiến trình đang chạy. Thông tin này sẽ được trả về cho user mode qua hệ thống gọi `sysinfo`.

- Thêm hàm `get_npoc` để đếm số tiến trình không ở trạng thái UNUSED vào file `kernel/proc.c`
- Thêm hàm `get_freemem` để tính bộ nhớ trống vào file `kernel/kalloc.c`
- Cập nhật file `kernel/defs.h`: khai báo hai hàm mới là `uint64 get_nproc(void)`; và `uint64 get_freemem(void)`; để các thành phần khác trong kernel có thể gọi và sử dụng các hàm này, ngay cả khi các hàm được định nghĩa trong các tệp `.c` khác.

C. TÀI LIỆU THAM KHẢO:

Sách xv6:

https://drive.google.com/file/d/1xSrasrtW3RQmAMLJO0jUNm13_7dxbYye/view